UNIVERSIDAD DON BOSCO

DISEÑO Y PROGRAMACIÓN DE SOFTWARE MULTIPLATAFORMA

DPS441 G01T

PROYECTO DE CATEDRA FASE 1

Alumnos:

. Edwin Daniel Leiva Barrera_____LB250368
· Kevin Jafet Aguilar Franco_____AF250113
· Cesar Armando Melendez Arias____MA250099
· Cesar Alejandro Lara Franco_____LL202677
· Anderson Alfredo Escobar Moran__EM251400

Docente:

. Emerson Francisco Cartagena Candelario

Índice

Introducción 4
Perfil del Proyecto – pideYcome 5
1. Nombre del Proyecto 5
2. Problema por resolver 5
3. Objetivo general 5
4. Metas 5
5. Resultados esperados 5
6. Metodología 6
7. Área geográfica y beneficiarios 6
8. Equipo participante 7
9. Fuentes de financiamiento 7
10. Factores críticos de éxito 7
Fase 1 – Análisis y Planificación (Semana 1) 8
Actividades 8
Fase 2 – Diseño del Sistema (Semana 2) 8
Actividades 8
Fase 3 – Desarrollo del Módulo de Autenticación (Semana 3) 8
Actividades 8
Fase 4 – Desarrollo del Módulo de Menú / Platillos (Semana 4 y 5) 9
Actividades 9
Fase 5 – Módulo de Pedidos y Gestión de Mesas (Semana 6 y 7) 9
Actividades 9
Fase 6 – Módulo Administrativo e Inventario (Semana 8) 9
Actividades 9
Fase 7 – Pruebas, Correcciones y Optimización (Semana 9) 9
Actividades 9
Fase 8 – Documentación y Entrega Final (Semana 10) 9
Actividades 10

Resumen del Cronograma 10
12. Presupuesto 11
Descripción detallada del funcionamiento propuesto 30
9. Herramientas y tecnologías a utilizar 45
Fase 1 – Análisis y Planificación (Semana 1) 52
Actividades 52
Fase 2 – Diseño del Sistema (Semana 2) 52
Actividades 52
Fase 3 – Desarrollo del Módulo de Autenticación (Semana 3) 52
Actividades 52
Fase 4 – Desarrollo del Módulo de Menú / Platillos (Semana 4 y 5) 53
Actividades 53
Fase 5 – Módulo de Pedidos y Gestión de Mesas (Semana 6 y 7) 53
Actividades 53
Fase 6 – Módulo Administrativo e Inventario (Semana 8) 53
Actividades 53
Fase 7 – Pruebas, Correcciones y Optimización (Semana 9) 54
Actividades 54
Fase 8 – Documentación y Entrega Final (Semana 10) 54
Actividades 54
Resumen del Cronograma 54

Introducción

En la actualidad, el uso de sistemas informáticos en los negocios se ha vuelto fundamental para mejorar la organización y eficiencia de los procesos. En el caso de los restaurantes, la implementación de un sistema permite optimizar la gestión de pedidos, el control de mesas, la comunicación entre el personal y el manejo de pagos.

El presente documento tiene como objetivo mostrar la planificación del desarrollo de un sistema para la gestión de un restaurante, el cual estará enfocado en cubrir las necesidades de distintos roles dentro del establecimiento, como el mesero, cocinero, cajero y administrador. Cada uno de estos usuarios contará con funciones específicas que permitirán mejorar el flujo de trabajo y reducir errores en la atención al cliente.

Además, se presenta el cronograma de actividades que se seguirá durante el desarrollo del proyecto, así como un presupuesto estimado que contempla los recursos humanos, tecnológicos y materiales necesarios para su realización. Esto permite tener una visión más clara del tiempo y costo que implica la implementación del sistema.

En general, este documento busca servir como una guía para el desarrollo del proyecto, organizando las etapas y recursos de manera estructurada para lograr un resultado funcional y eficiente.

Perfil del Proyecto – pideYcome

Mes y año de elaboración: Marzo 2026

1. Nombre del Proyecto

pideYcome

2. Problema por resolver

Los restaurantes enfrentan problemas de organización como gestión ineficiente de pedidos, retrasos entre áreas operativas, falta de control de inventarios y ausencia de información consolidada. El proyecto busca digitalizar y optimizar estos procesos para mejorar la eficiencia.

3. Objetivo general

Desarrollar un sistema multiplataforma que gestione de forma eficiente los procesos operativos de un restaurante, integrando módulos clave como pedidos, menú, inventarios y administración.

4. Metas

- Implementar un módulo de autenticación seguro.
- Digitalizar la toma de ordenes.
- Administrar el menú mediante CRUD.
- Crear panel administrativo web.
- Automatizar notificaciones y manejo de errores.
- Crear UI basada en prototipo de Figma.
- Entrega de MVP funcional antes del cierre del ciclo.

5. Resultados esperados

- Reducción del tiempo de atención.
- Mejor coordinación entre áreas.
- Eliminación de errores manuales.
- Control preciso de inventario.
- Historial de ventas y pedidos.
- Plataforma escalable.
- Mejor experiencia del cliente.

6. Metodología

Para el desarrollo del proyecto se utilizará un enfoque ágil basado en **Kanban**, gestionado mediante **Trello**. Esta metodología permite visualizar el trabajo, priorizar tareas y mantener un flujo continuo de desarrollo sin necesidad de ciclos rígidos.

El tablero de Trello se organizará con las siguientes columnas:

- **Backlog:** contiene todas las tareas y funcionalidades pendientes por desarrollar.
- **To Do:** tareas seleccionadas para trabajar durante la semana.
- **In Progress:** actividades que se están desarrollando activamente.
- **Review / Testing:** tareas terminadas que están siendo revisadas y probadas.
- **Done:** actividades completadas y validadas.

Durante el proyecto se aplicarán prácticas clave de Kanban como:

- **Límites de trabajo en progreso (WIP)** para evitar sobrecarga.
- **Avance incremental**, entregando pequeñas funcionalidades completas.
- **Revisiones semanales**, donde se evalúan avances, bloqueos y prioridades.
- **Documentación dentro de Trello**, incluyendo checklists, adjuntos, enlaces a commits y notas.

Este enfoque garantiza organización visual, flexibilidad, rapidez en la toma de decisiones y un flujo de trabajo ordenado para completar el sistema de restaurante de forma eficiente.

7. Área geográfica y beneficiarios

El proyecto se orienta a restaurantes ubicados en **El Salvador**, con posibilidad de ampliar su uso a la región centroamericana.
Los beneficiarios directos serán:

- Administradores del restaurante.
- Personal de cocina.
- Meseros y cajeros.
- Clientes, al recibir un mejor servicio.

8. Equipo participante

- ✓ Cesar Alejandro Lara Franco – Desarrollador Principal / Analista
- ✓ Anderson Alfredo Escobar Morán – Diseñadora UI/UX (encargada de prototipos en Figma)
- ✓ Edwin Daniel Leiva Barrera – Desarrollador Frontend (implementación de interfaces y componentes)
- ✓ Cesar Armando Melendez Arias – Desarrolladora Backend (APIs, lógica de negocio y base de datos)
- ✓ Kevin Jafet Aguilar Franco.– Tester / QA (pruebas funcionales, reportes de errores y validación del sistema)

9. Fuentes de financiamiento

- Inversión del restaurante.
- Fondos académicos.
- Patrocinios tecnológicos.
- Microfinanciamiento para digitalización.

10. Factores críticos de éxito

Riesgos:

- Retrasos en desarrollo por complejidad de módulos.
- Problemas de integración entre frontend y servicios backend.
- Falta de dispositivos adecuados en el restaurante.
- Capacitación insuficiente del personal.

Soluciones:

- Desarrollar por sprints claros y medibles.
- Realizar pruebas estrictas por módulo.
- Asegurar compatibilidad multiplataforma.
- Documentación clara y capacitación simple para usuarios finales.

11. Duración

Fase 1 – Análisis y Planificación (Semana 1)

Actividades

- Revisión del problema y requerimientos funcionales
- Definición del alcance del sistema
- Análisis del prototipo de Figma
- Configuración del tablero Kanban en Trello
- Creación del backlog inicial de tareas

Duración: 1 semana

Fase 2 – Diseño del Sistema (Semana 2)

Actividades

- Ajustes del prototipo de Figma según requerimientos
- Arquitectura del sistema (componentes React, Context API, rutas)
- Diseño del flujo de navegación (router)
- Definición del modelo de datos y endpoints de API

Duración: 1 semana

Fase 3 – Desarrollo del Módulo de Autenticación (Semana 3)

Actividades

- Implementación del *AuthProvider*
- Pantallas: login, registro, recuperación
- Validaciones y control de acceso
- Pruebas iniciales del flujo de autenticación

Duración: 1 semana

Fase 4 – Desarrollo del Módulo de Menú / Platillos (Semana 4 y 5)

Actividades

- CRUD de platillos (crear, listar, editar, eliminar)
- Administración de categorías
- Integración visual del diseño de Figma
- Pruebas funcionales del módulo

Duración: 2 semanas

Fase 5 – Módulo de Pedidos y Gestión de Mesas (Semana 6 y 7)

Actividades

- Registro de pedidos
- Envío de pedidos a cocina
- Estado del pedido (pendiente, en preparación, entregado)
- Control de mesas ocupadas / disponibles
- Integración con notificaciones (*Toaster*)

Duración: 2 semanas

Fase 6 – Módulo Administrativo e Inventario (Semana 8)

Actividades

- Panel de administración
- Gestión de usuarios y roles
- Control de inventario básico
- Generación de reportes simples

Duración: 1 semana

Fase 7 – Pruebas, Correcciones y Optimización (Semana 9)

Actividades

- Pruebas completas del sistema
- Corrección de errores
- Optimización de componentes
- Revisión del manejo de errores con *ErrorBoundary*
- Revisión de flujos en Trello y documentación de tareas

Duración: 1 semana

Fase 8 – Documentación y Entrega Final (Semana 10)

Actividades

- Preparación del documento final del proyecto
- Capturas del sistema y evidencia en Trello
- Elaboración de la presentación final
- Ensayo de defensa del proyecto

Duración: 1 semana

Resumen del Cronograma

Fase	Actividades principales	Duración
1	Análisis y Planificación	1 semana
2	Diseño del Sistema	1 semana
3	Módulo de Autenticación	1 semana
4	Módulo de Menú	2 semanas
5	Pedidos y Mesas	2 semanas
6	Panel Admin e Inventario	1 semana
7	Pruebas y Optimización	1 semana
8	Documentación	1 semana

Duración Total: 10 Semanas

12. Presupuesto

Recursos Humanos

Rol	Actividad	Horas estimadas	Costo por hora	Subtotal
Desarrollador Principal	Implementación general, integración, pruebas	40 h	$10/h	**$400**
Diseñador UI/UX	Prototipado en Figma, diseño de pantallas	10 h	$8/h	**$80**
Desarrollador Frontend	Componentes, rutas, UI React	20 h	$9/h	**$180**
Desarrollador Backend	API, base de datos, lógica	20 h	$10/h	**$200**
Tester / QA	Pruebas funcionales y reporte de errores	10 h	$7/h	**$70**

✓ **Subtotal Recursos Humanos: $930**

Infraestructura Tecnológica

Recurso	Detalle	Costo mensual	Meses	Subtotal
Hosting (VPS o Shared)	Para backend y base de datos	$10	1	**$10**
Base de datos (si se usa servicio externo tipo Railway/Supabase)	Plan básico	$5	1	**$5**
Dominio (opcional)	.com o .sv	$12 anual	Proporción 1 mes ($1)	**$1**

✓ **Subtotal Infraestructura: $16**

Software y Herramientas

Herramienta	Función	Costo
Trello	Gestión Kanban del proyecto	**$0 (plan gratuito)**
GitHub	Control de versiones y repositorio	**$0**
Figma	Diseño UI/UX	**$0**
Visual Studio Code	Entorno de desarrollo	**$0**

✓ **Subtotal Software: $0**

Equipos y Materiales

Material o equipo	Uso	Costo aproximado
Energía eléctrica y datos (proporcional)	Desarrollo, pruebas	**$10**
Material de apoyo / impresiones	Documentos del proyecto	**$5**

✓ **Subtotal Equipos y Materiales: $15**

Contingencias (10%)

Para cubrir imprevistos técnicos o extensiones de módulos:

- **10% del total parcial ($930 + $16 + $15) = $96**

✓ **Subtotal Contingencias: $96**

Resumen General del Presupuesto

Categoría	Subtotal
Recursos Humanos	$930
Infraestructura	$16
Software	$0
Equipos y Materiales	$15

Contingencias	$96

TOTAL, ESTIMADO DEL PROYECTO: $1,057 USD

Metodología de Trabajo del Equipo

Enfoque utilizado: Kanban

Para la organización y gestión del desarrollo del proyecto se utiliza el enfoque **Kanban**, el cual permite visualizar el trabajo, optimizar el flujo de tareas y mantener un proceso continuo e incremental. Este método facilita identificar cuellos de botella, controlar la carga del equipo mediante límites de trabajo en progreso (WIP) y priorizar actividades según el valor que aportan al producto mínimo viable (MVP).

El tablero Kanban gestionado en **Trello** sirve como herramienta principal para:

- Registrar todas las tareas.
- Controlar el progreso.
- Dar seguimiento a cada fase del proyecto.
- Mantener evidencia del avance y la colaboración del equipo.

Definición de roles de cada integrante

El equipo de desarrollo del proyecto está conformado por los siguientes roles:

1. Coordinador del Proyecto — Cesar Alejandro Lara Franco

- Supervisar el cumplimiento de las fases.
- Organizar el flujo de trabajo en Trello.
- Verificar avances y asignar tareas.
- Consolidar documentación y cronograma.
- Responsable de integración final del sistema.

2. Diseñadora UI/UX — Anderson Alfredo Escobar Morán

- Diseño del prototipo en Figma.
- Definición de estilos visuales.
- Apoyo en revisión de componentes UI.

- Entrega de guías visuales y flujos de navegación.

3. Desarrollador Frontend — Edwin Daniel Leiva Barrera

- Implementación de interfaces en React.
- Integración con Context API.
- Desarrollo de pantallas del menú, pedidos y administración.
- Asegurar la adaptación a prototipo UI.

4. Desarrolladora Backend — Cesar Armando Melendez Arias

- Diseño del modelo de datos.
- Creación de endpoints y lógica del servidor.
- Integración con frontend.
- Gestión de base de datos e inventario.

5. Tester / QA — Kevin Jafet Aguilar Franco

- Diseño y ejecución de pruebas funcionales.
- Registro de errores y seguimiento en Trello.
- Validación final de cada módulo completado.
- Asegurar calidad del producto.

Evidencia del uso de Trello

i. Descripción de la organización del tablero

El tablero del proyecto se estructura siguiendo el flujo estándar de Kanban:

1. **Backlog**
    - Lista completa de funcionalidades pendientes.
    - Incluye tareas grandes divididas en subtareas.
2. **To Do (Por hacer)**
    - Tareas seleccionadas para la semana en curso.
    - Priorización semanal por el coordinador.

3. **In Progress (En progreso)**
    - Actividades que se están desarrollando actualmente.
    - Se aplican límites WIP (máximo 2 tareas por integrante).
4. **Review / Testing**
    - Tareas completadas que requieren revisión por QA.
    - Validación de funcionalidad, UI y lógica.
5. **Done (Completado)**
    - Actividades finalizadas y verificadas.
    - Incluye evidencia, comentarios y checklist cumplido.

Cada tarjeta del tablero contiene:

- Descripción detallada de la tarea.
- Checklist de subtareas.
- Responsable asignado.
- Etiquetas por módulo (auth, menú, pedidos, UI/UX, admin).
- Fechas límite.
- Archivos adjuntos (capturas, código, enlaces).

ii. Enlace público al espacio de trabajo

Enlace al tablero Trello:
https://trello.com/invite/b/69b8a172ddbce1671e34755a/ATTI578335f2112471151b212d9e1b68cf33C7E714C7/gestion-del-proyecto-fase-1

iii. Planificación de actividades y seguimiento

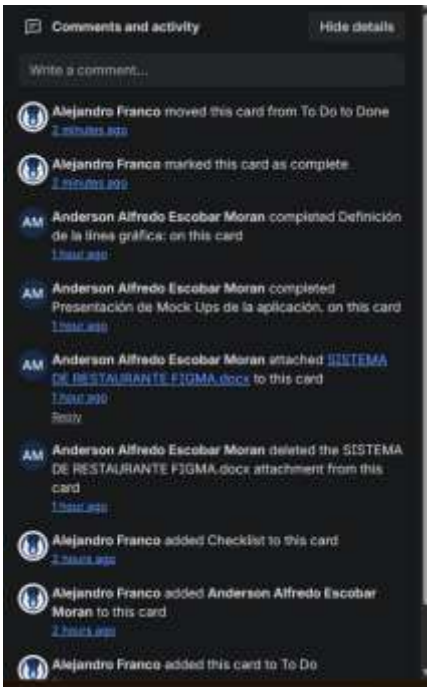
La planificación del trabajo se realiza en base al cronograma definido previamente:

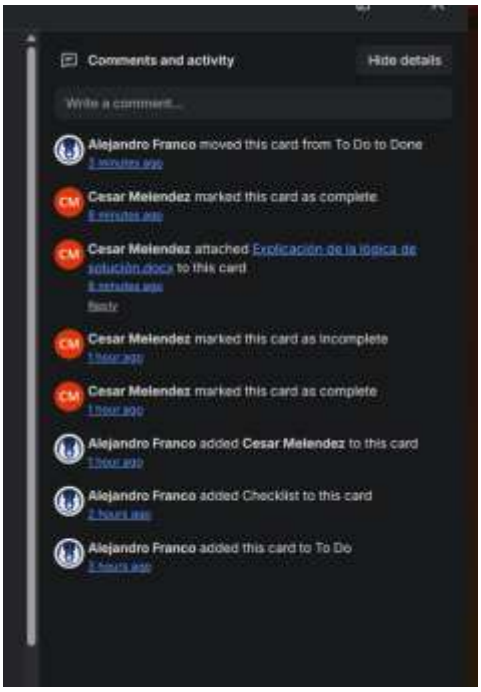
- Cada semana se seleccionan tareas del Backlog y se mueven a **To Do**.
- Durante el desarrollo, se desplazan a **In Progress**.
- Al finalizarse, pasan a **Review / Testing**, donde el Tester valida la calidad.

- Una vez aprobadas, se trasladan a **Done**.

El seguimiento incluye:

- **Revisiones semanales del avance**.
- **Comentarios dentro de cada tarjeta** para registrar decisiones.
- **Historial de tareas completadas** como evidencia para el informe del proyecto.
- **Control visual del progreso** del equipo mediante el tablero Kanban.





Sistema de Gestión de Restaurante (PideyCome)

Descripción General

Sistema completo de gestión de restaurante desarrollado como mockup de prueba con múltiples roles de usuario. Permite la gestión integral de órdenes, cocina, pagos e inventario con actualizaciones en tiempo real.

Roles de Usuario

1. Mesero

- Crea órdenes para llevar o comer en el lugar.
- Asigna mesas a las órdenes.
- Agrega nombre del cliente.
- Recibe notificaciones en tiempo real del estado de órdenes.
- Ve campanita de notificaciones con contador de pendientes.
- Filtra órdenes por mesa.
- Selecciona productos del menú (los agotados aparecen deshabilitados).

2. Cocina

- Ve todas las órdenes en tiempo real.
- Estados: Pendiente → Recibida → Preparando → Despachada.
- Filtra por estado de orden.
- Filtra por mesa o "para llevar".
- Ve ID de mesa o identificador para llevar.
- Ve nombre del cliente en cada orden.
- Actualiza el estado de las órdenes.

3. Cajero

- Ve recuento automático de órdenes.
- Calcula totales automáticamente.
- Agrupa órdenes por mesa.
- Imprime tickets o facturas.
- Simulación de POS.
- Envío de comprobantes por correo.
- Gestiona pagos y marca órdenes como pagadas.
- Ve ingresos del día y estadísticas.

4. Administrador

- Gestiona productos, precios e inventario.
- Ve todos los movimientos del restaurante.
- Crea nuevos usuarios (mesero, cocina, cajera).
- Agrega nuevos productos al inventario.
- Actualiza stock de productos.
- Modifica precios y categorías.
- Panel de estadísticas general.

Usuarios de Prueba

(Los datos de prueba no están insertados en ninguna base de datos)

Rol	Usuario	Contraseña
Mesero	mesero1	1234
Cocina	cocina1	1234
Cajera	cajera1	1234
Administrador	admin	admin

Flujo de Estados de Órdenes

Ordenada → Recibida por Cocina → Preparando → Despachada

Estados Detallados:

1. **Ordenada (Amarillo):** El mesero crea la orden.
2. **Recibida por Cocina (Azul):** La cocina confirma recepción.
3. **Preparando (Naranja):** La orden está en preparación.
4. **Despachada (Verde):** La orden está lista para entregar.

Funcionalidades Principales

Sistema de Órdenes

- Creación de órdenes con nombre de cliente.
- Asignación de mesa o "para llevar".
- Carrito de compras con cantidades ajustables.
- Cálculo automático de totales.
- Actualización en tiempo real de estados.
- Notificaciones push para meseros.
- Filtrado por mesa y estado.
- Historial completo de órdenes.

Gestión de Productos

- Catálogo de productos por categorías.
- Control de stock en tiempo real.
- Productos agotados visibles pero deshabilitados.
- Indicador de stock bajo (menos de 10 unidades).
- Agregar nuevos productos desde admin.
- Modificación de precios e inventario.
- Imágenes de productos.

Sistema de Notificaciones

- Campanita de notificaciones para meseros.
- Contador de órdenes pendientes.
- Actualización en tiempo real.
- Indicador visual con badge rojo.
- Lista desplegable de notificaciones recientes.

Punto de Venta (POS)

- Agrupación de órdenes por mesa.
- Cálculo automático de totales.
- Opciones de pago: Efectivo, Tarjeta.
- Generación de tickets y facturas.
- Envío por correo electrónico.
- Simulación de impresión.
- Marcado de órdenes como pagadas.

Panel Administrativo

- Gestión completa de usuarios.
- Creación de nuevos usuarios (sin rol admin).
- Gestión de inventario expandible.
- Actualización de precios y stock.
- Estadísticas de ventas.
- Historial de movimientos.
- Panel de control general.

Recuperación del Sistema

- Persistencia de datos en localStorage.
- Recuperación automática al recargar.

- Manejo de errores y estados vacíos.
- Validaciones de formularios.

Diseño UX/UI

Paleta de Colores

Color Principal - Naranja

- **Primario**: orange-500 (#f97316).
- **Hover**: orange-600 (#ea580c).
- **Degradados**: from-orange-50 to-orange-100.
- **Uso**: Botones principales, iconos de navegación, precios, elementos destacados.

Colores de Estado de Órdenes

- **Amarillo** (yellow-500) - Estado "Ordenada".
- **Azul** (blue-500/600) - Estado "Recibida por Cocina".
- **Naranja** (orange-500/600) - Estado "Preparando".
- **Verde** (green-500/600) - Estado "Despachada".

Colores por Rol

- **Mesero**: Naranja (orange-500).
- **Cocina**: Naranja (orange-500).
- **Caja**: Verde (green-500/600).
- **Admin**: Sistema por defecto.
-

Colores de Información

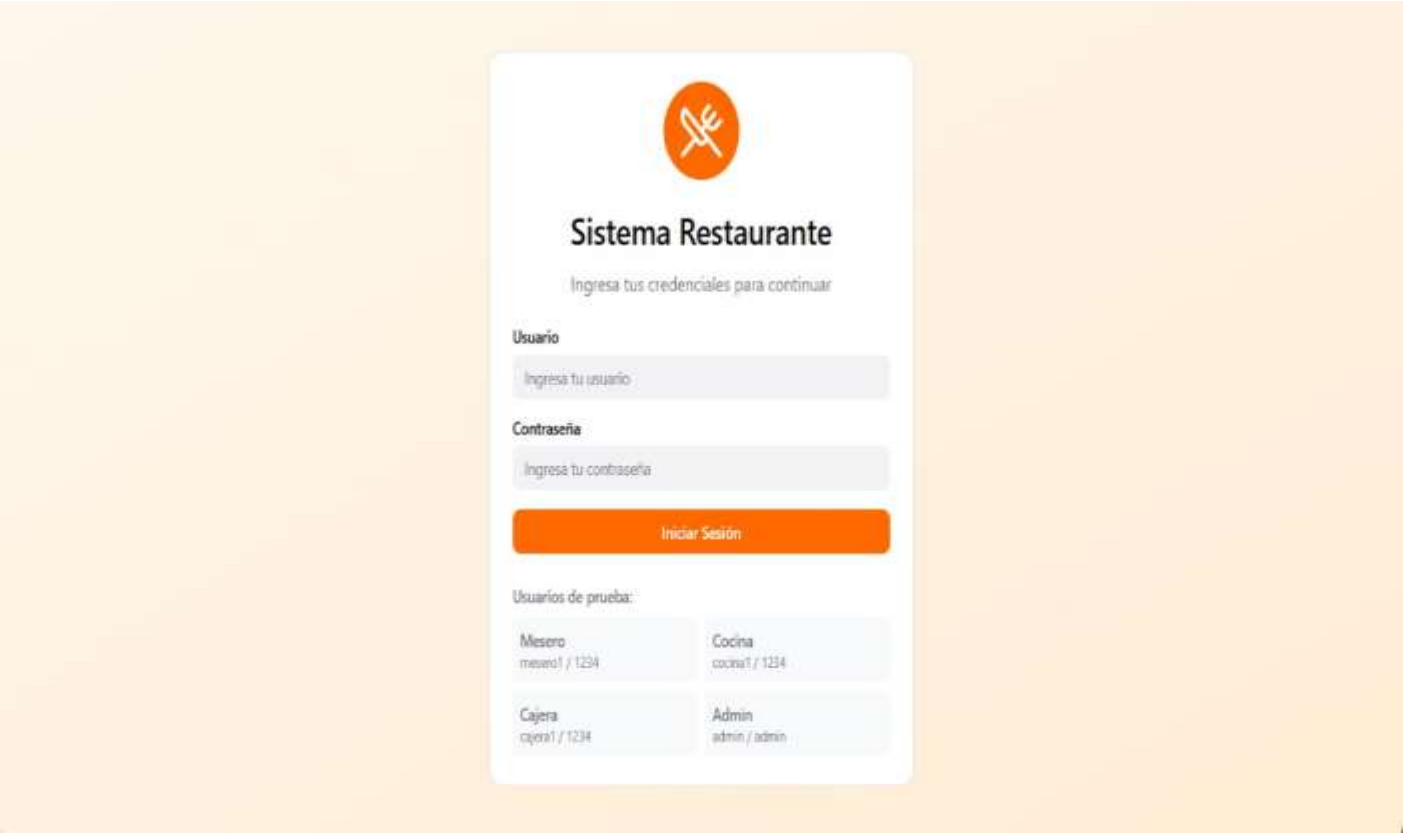
- **Verde** (green-600) - Ingresos, órdenes pagadas, confirmaciones.
- **Azul** (blue-600) - Contador de tickets, información general.
- **Naranja** (orange-600) - Órdenes pendientes de pago, alertas.
- **Rojo** (red-500) - Notificaciones urgentes, contador de pendientes.

Colores de Base (Sistema de Diseño)

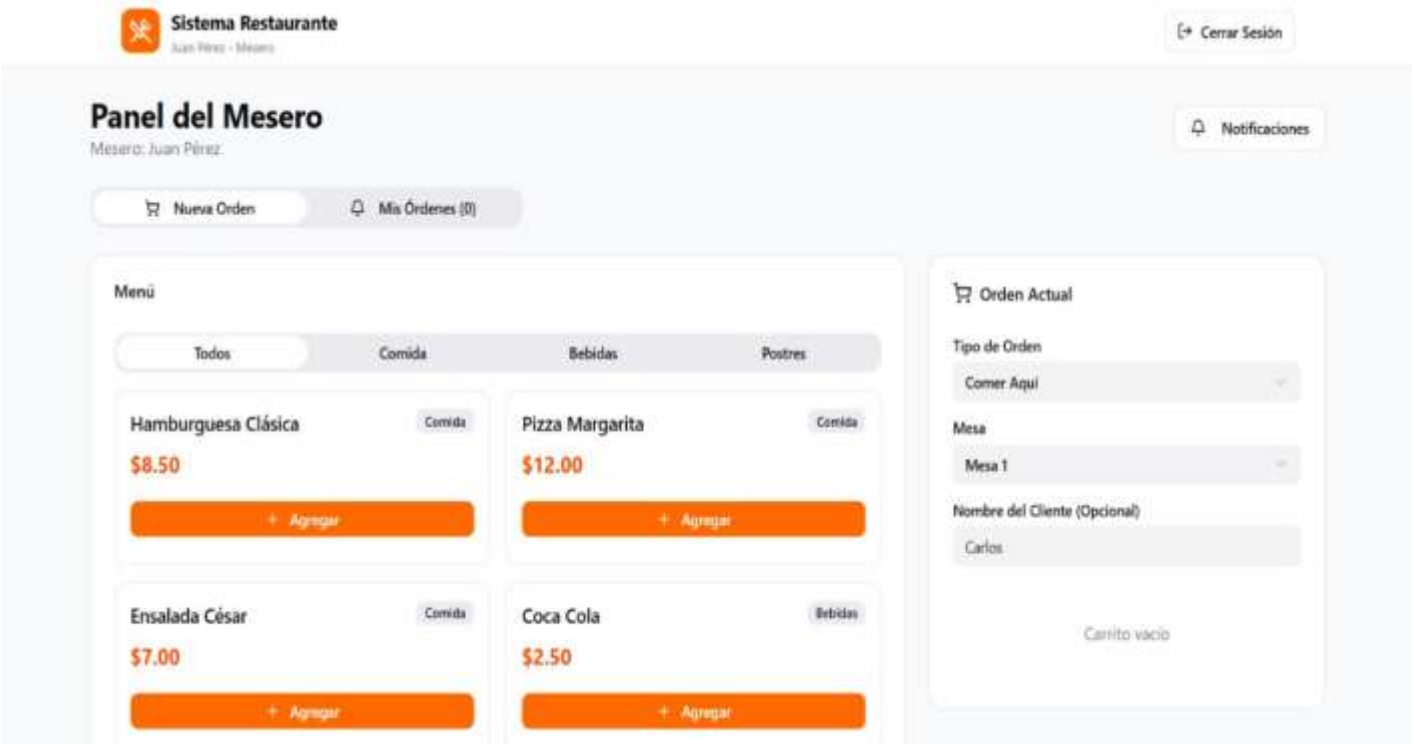
- **Background**: #ffffff (blanco).
- **Foreground**: oklch(0.145 0 0) (negro azulado).
- **Primary**: #030213 (negro azulado).
- **Destructive**: #d4183d (rojo).
- **Muted**: #ececf0 (gris claro).
- **Border**: rgba(0, 0, 0, 0.1) (gris transparente).

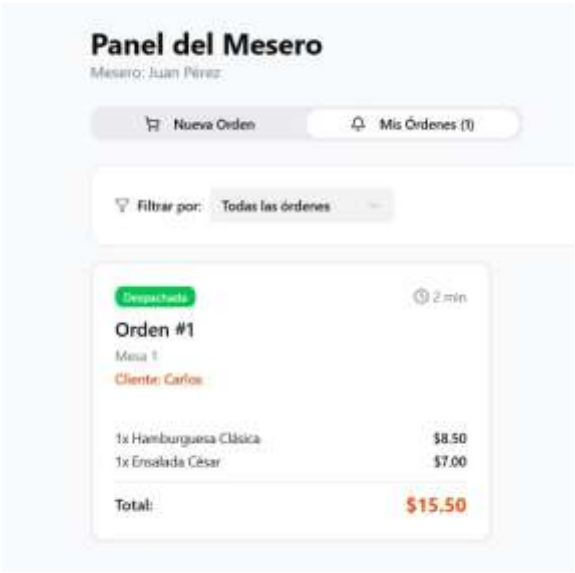
Imágenes del prototipo

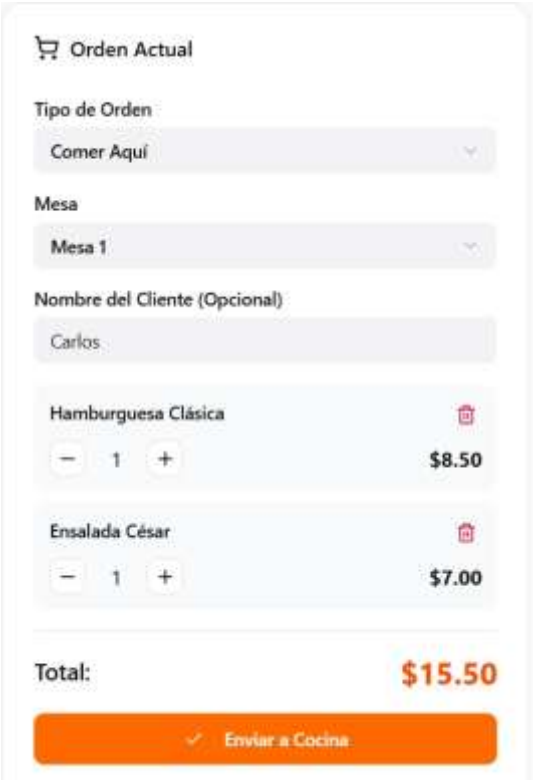
Login donde se ingresará el usuario y contraseña



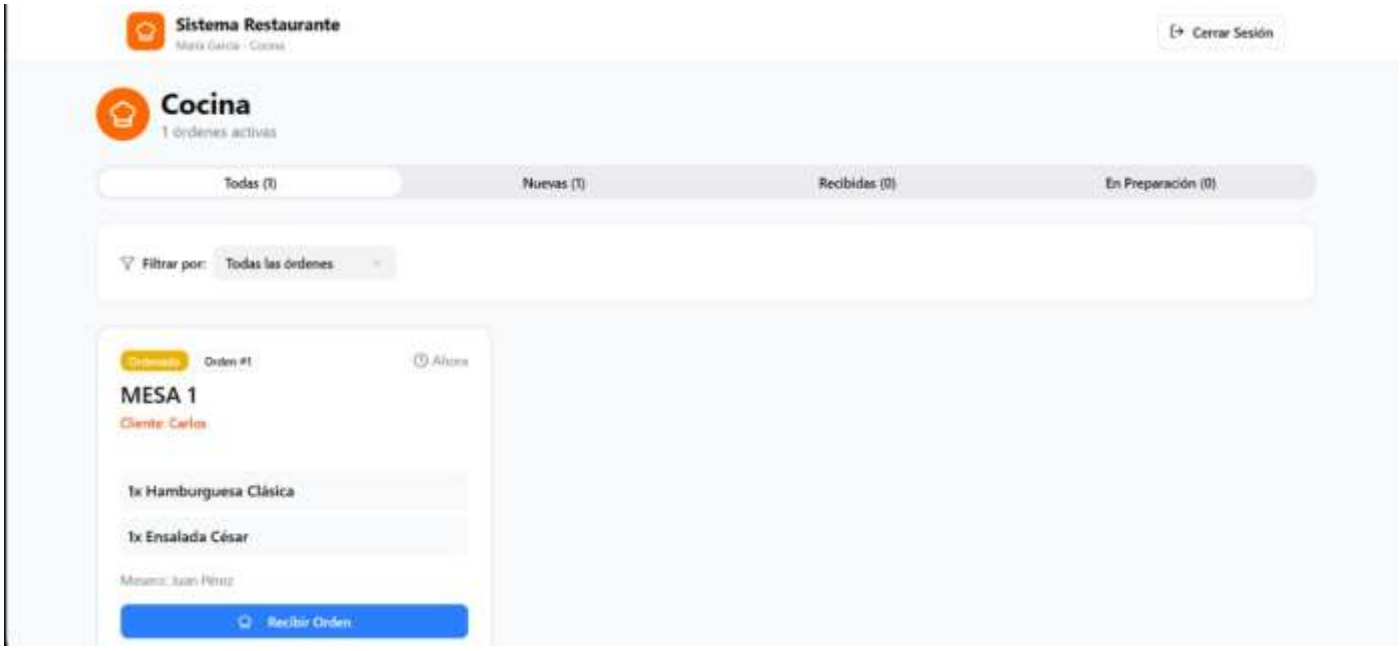
Capturas de pantalla del perfil mesero:



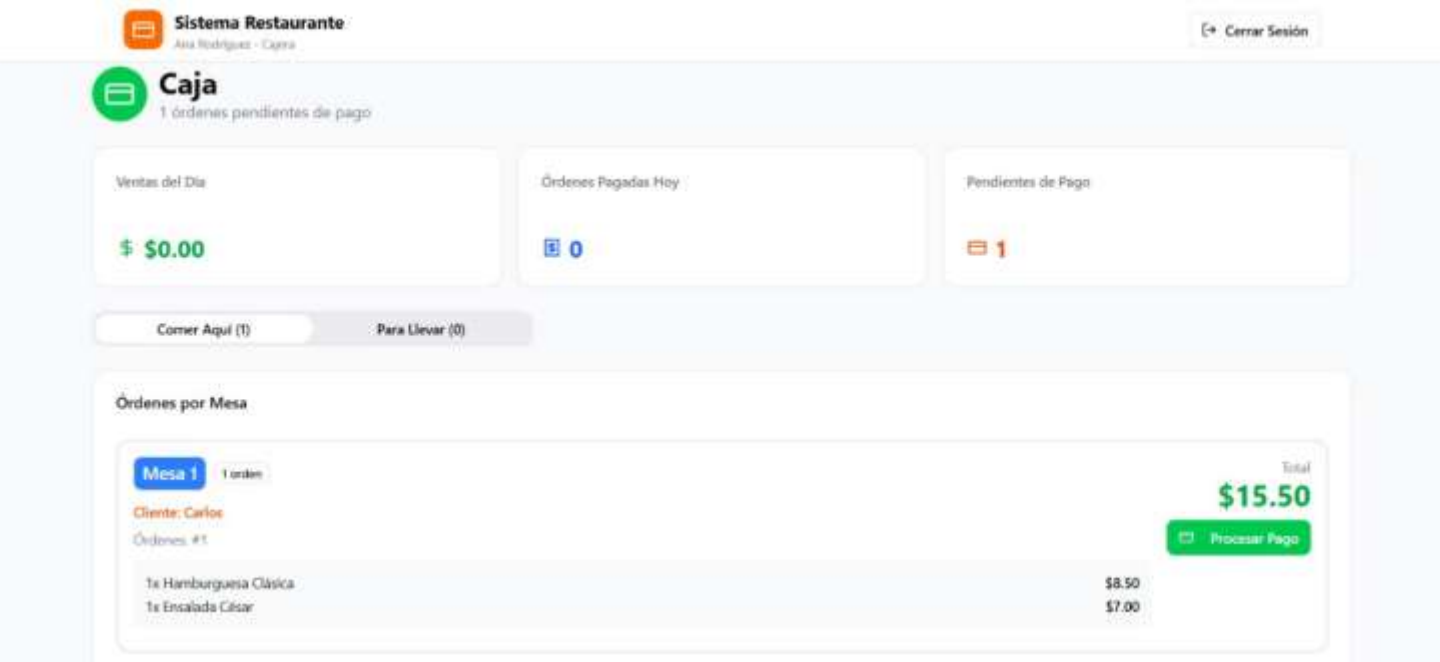


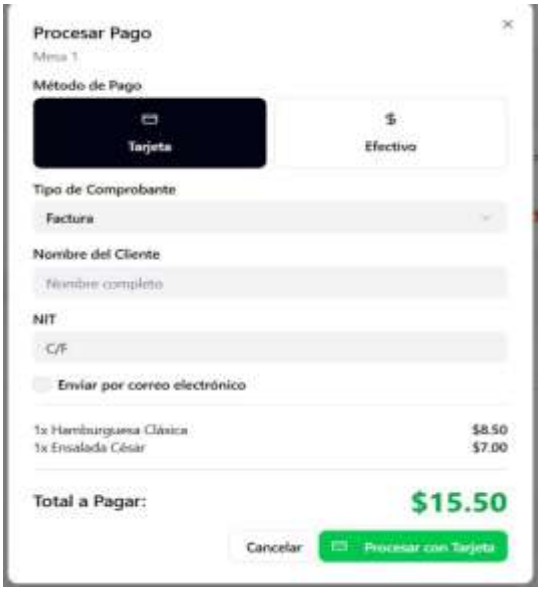


Capturas de pantalla del perfil cocina:



Capturas de pantalla del perfil cajero:

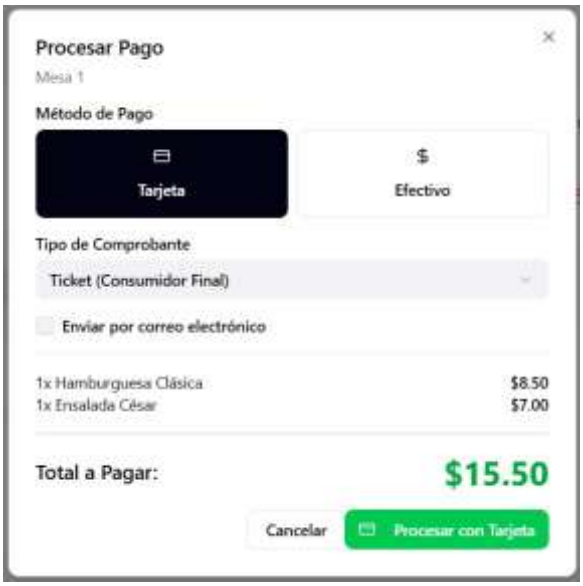


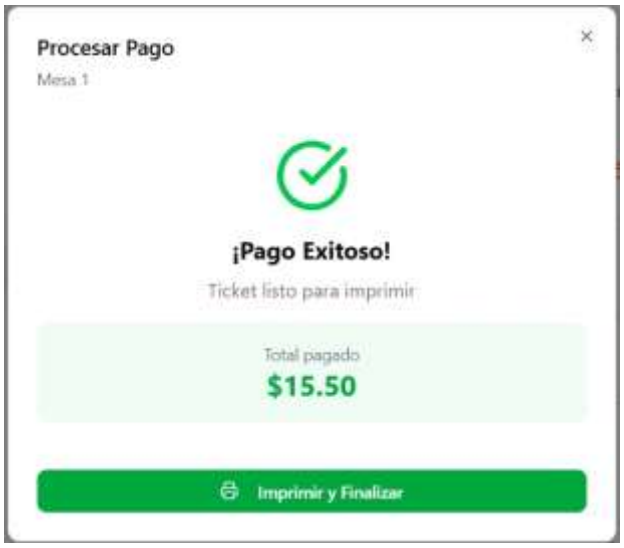


☑ Enviar por correo electrónico

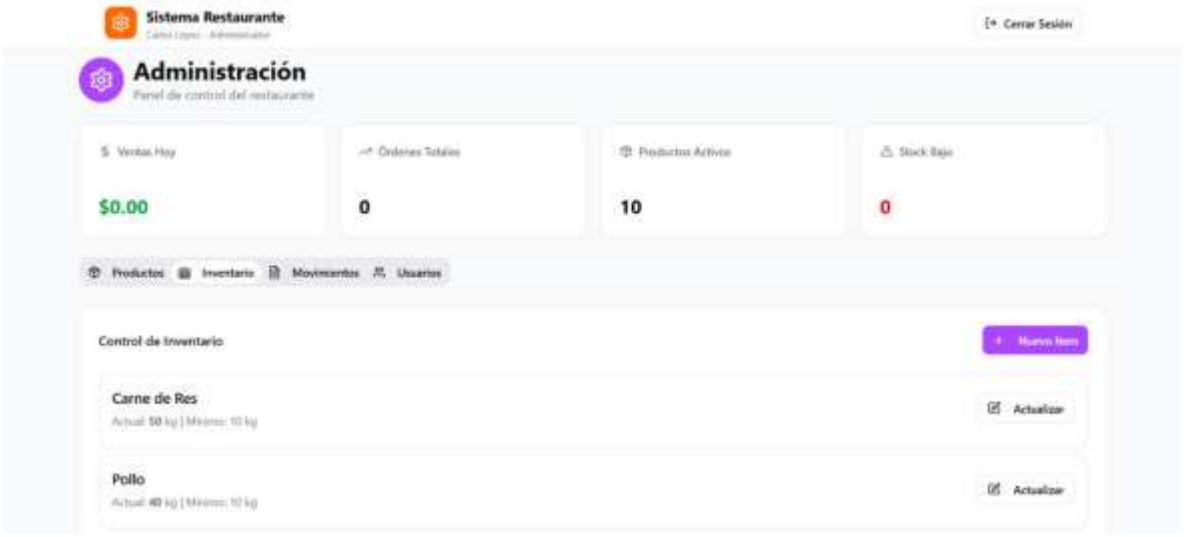
Correo Electrónico

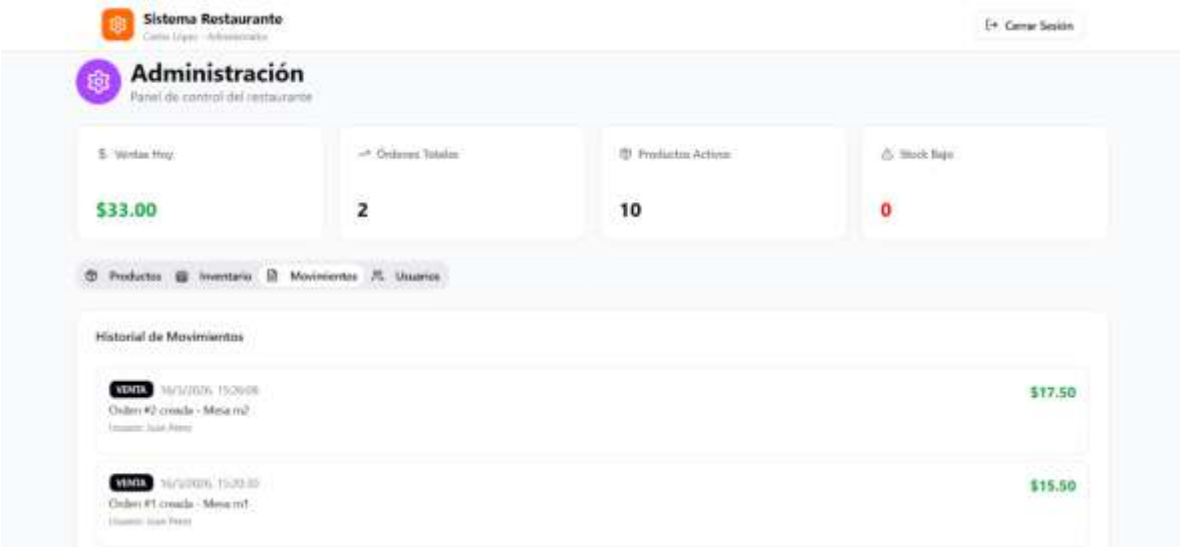
cliente@ejemplo.com





Capturas de pantalla del perfil administrador:



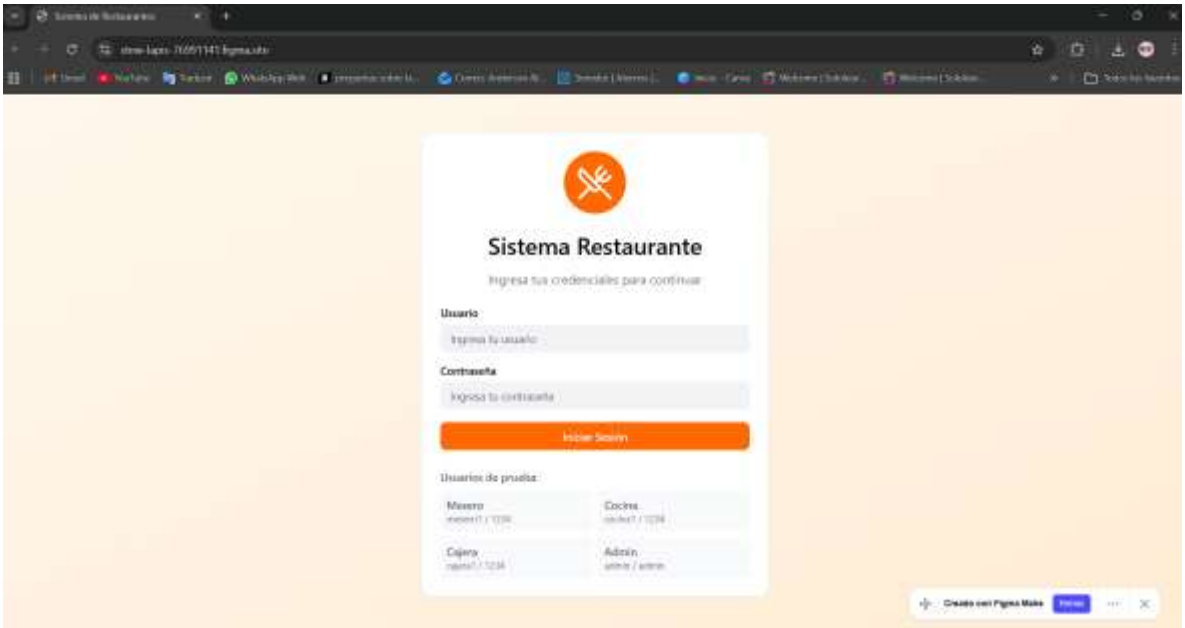


Link para visualizarlo personalmente en diferentes dispositivos desde el navegador.

https://stew-lapis-76991141.figma.site/

Figma nos da la opción de poder subir los prototipos o Mockups en línea para simular como quedaría una página web ya funcional.

Y debería de salir así:



Tecnologías Utilizadas

(Tecnologías temporales ya que usaremos JavaScript en reemplazo de TypeScript)

Frontend

- **React** - Biblioteca de UI.
- **TypeScript** - Tipado estático.
- **React Router** - Navegación y routing.
- **Tailwind CSS v4** - Framework de estilos.
- **Lucide React** - Iconografía.

Componentes UI

- Custom UI components basados en Radix UI.
- Card, Button, Input, Select, Checkbox.
- Dialog, Tabs, Badge, Label.
- Sheet (drawer mobile).

Notas Importantes

Mockup de Prueba

Este sistema es un **mockup de demostración** y no lo utilizaremos en el desarrollo del software real:

- Los datos se almacenan localmente en el navegador.
- No hay autenticación real ni seguridad.
- No hay conexión a base de datos.
- Las contraseñas son de ejemplo.
- La impresión y envío de correos son simulados.

Uso Recomendado

Este sistema es ideal para demostraciones de concepto, prototipos de UI/UX y presentaciones a clientes.

Próximas Mejoras Potenciales

- Integración con backend real como (Java Spring Boot).
- Base de datos MySQL.
- Autenticación JWT.
- WebSockets para tiempo real verdadero.

Desarrollado como sistema mockup de gestión de restaurante (PideyCome) | Grupo 2 DPS 01T 2026

Descripción detallada del funcionamiento propuesto

El sistema de gestión de restaurante propuesto es una solución integral diseñada para automatizar, organizar y optimizar los procesos operativos dentro de un restaurante, integrando en una sola plataforma las áreas de atención al cliente, cocina, caja y administración.

El sistema funciona mediante una arquitectura centralizada donde toda la información se gestiona en tiempo real, permitiendo que cada rol interactúe con los datos según sus permisos y responsabilidades.

Funcionamiento general del sistema

El flujo del sistema inicia con la creación de una orden por parte del mesero y finaliza con el procesamiento del pago por parte de la cajera, pasando por un control de estados gestionado por cocina.

Flujo general

1. El mesero crea una orden
2. La orden se registra en el sistema
3. Cocina recibe la orden en tiempo real
4. Cocina actualiza el estado de la orden

5. El sistema notifica al mesero
6. La cajera procesa el pago
7. El administrador supervisa todo el proceso

Módulo de mesero

El mesero es el encargado de registrar las órdenes de los clientes.

Funcionalidades:

- Ingresa el **nombre del cliente (obligatorio)**
- Define si la orden es:
    - Para mesa
    - Para llevar
- Selecciona productos del menú
- Visualiza:
    - Productos disponibles
    - Productos agotados (deshabilitados)
    - Productos con bajo stock (indicador "Quedan X")
- Maneja un carrito de compras con cantidades
- Genera la orden

Funcionamiento

Al confirmar la orden:

- Se crea un registro en la tabla ordenes
- Se agregan productos en detalle orden
- Se registra el estado inicial: **Ordenada**
- Se genera una notificación para cocina

Módulo de cocina

La cocina se encarga de gestionar el procesamiento de las órdenes.

Funcionalidades

- Visualiza todas las órdenes en tiempo real
- Filtra por:
  - Estado
  - Mesa o para llevar
- Observa:
  - Nombre del cliente
  - Mesa
  - Productos solicitados
- Cambia el estado de las órdenes mediante botones

Flujo de estados

- Ordenada → Recibida
- Recibida → Preparando
- Preparando → Despachada

Funcionamiento

Cada cambio de estado:

- Se actualiza en la tabla ordenes
- Se registra en historial estados
- Se genera una notificación al mesero

Módulo de caja (cajera)

La cajera es responsable del cobro y registro de ingresos.

Funcionalidades

- Visualiza órdenes **despachadas**
- Agrupa órdenes por mesa
- Calcula automáticamente los totales
- Selecciona:
    - Tipo de comprobante (ticket o factura)
    - Método de pago (efectivo, tarjeta, transferencia)
- Simula impresión de ticket
- Marca órdenes como pagadas

Funcionamiento

Cuando se procesa un pago:

- Se registra en la tabla pagos
- Se actualiza el estado de la orden a pagada
- Se reflejan los ingresos en estadísticas

MÓDULO DE ADMINISTRADOR

El administrador tiene control total del sistema.

Funcionalidades

- Gestión de productos:
    - Crear
    - Editar
    - Actualizar stock
- Gestión de usuarios:

    - Crear nuevos usuarios (excepto admin)
- Visualización de estadísticas:
    - Ingresos
    - Órdenes
    - Actividad del sistema
- Registro de movimientos

Funcionamiento

Cada acción del administrador:

- Se guarda en la tabla movimientos
- Permite auditoría completa del sistema

sistema de notificaciones

El sistema incluye notificaciones en tiempo real para mejorar la comunicación.

Funcionamiento

- Cada cambio de estado genera una notificación
- Se almacena en la tabla notificaciones
- El mesero recibe alertas mediante una campanita
- Se muestra contador de notificaciones pendientes

manejo de inventario

El sistema controla el stock de productos.

Funcionamiento

- Cada producto tiene un campo stock

- Si stock = 0 → producto agotado (no seleccionable)
- Si stock < 10 → alerta visual "Quedan X"
- El administrador puede actualizar el inventario

persistencia y recuperación

El sistema garantiza la integridad de los datos.

Funcionamiento

- Los datos se almacenan en:
    - Base de datos (backend)
    - localStorage (frontend)
- Permite recuperación automática en caso de fallo
- Mantiene el estado al recargar la aplicación

interfaz y experiencia de usuario

El sistema está diseñado para ser intuitivo y accesible.

Características

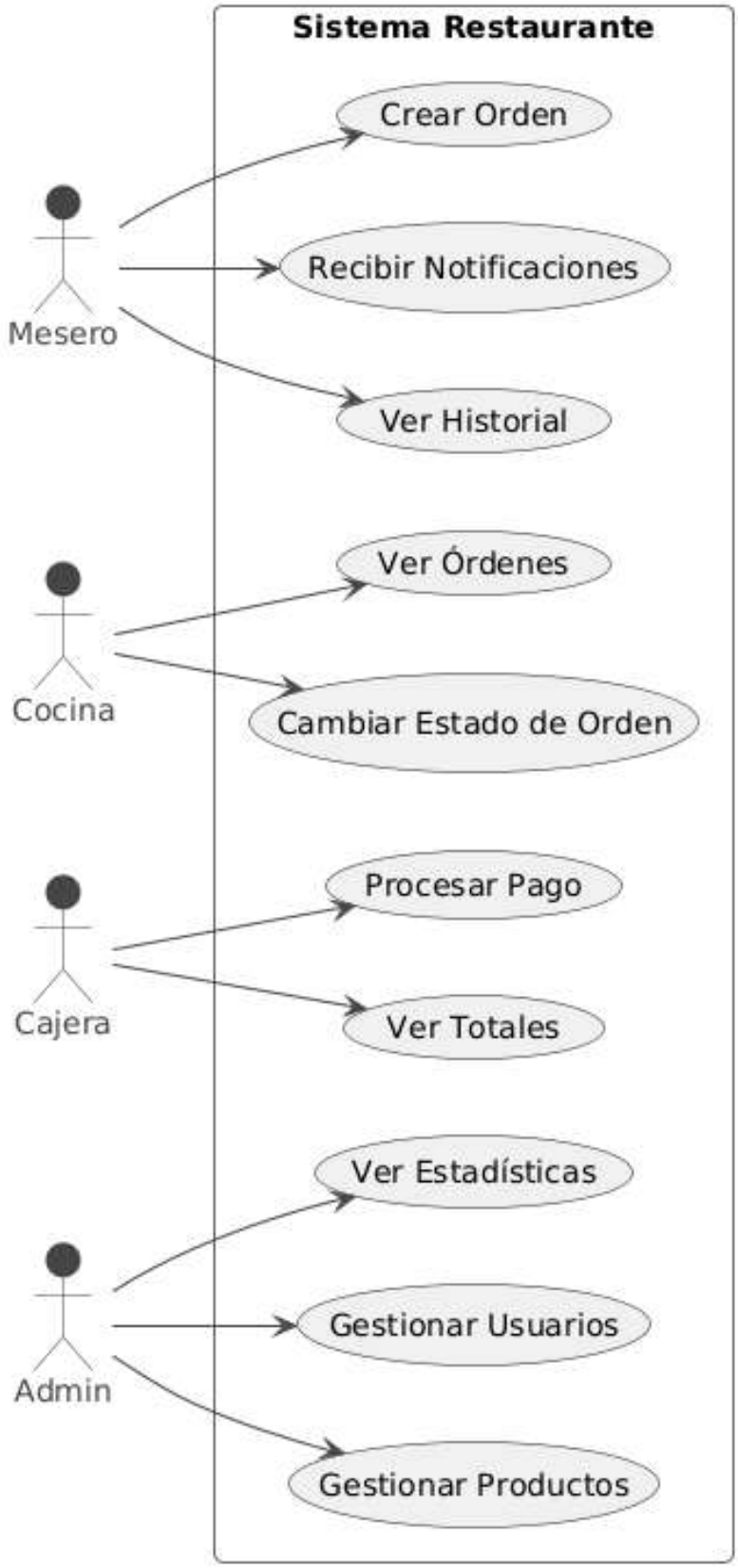
- Diseño responsivo (móvil, tablet, desktop)
- Uso de colores para estados:
    - Ordenada
    - Recibida
    - Preparando
    - Despachada
- Iconografía clara (lucide-react)
- Navegación simple por roles

Conclusión

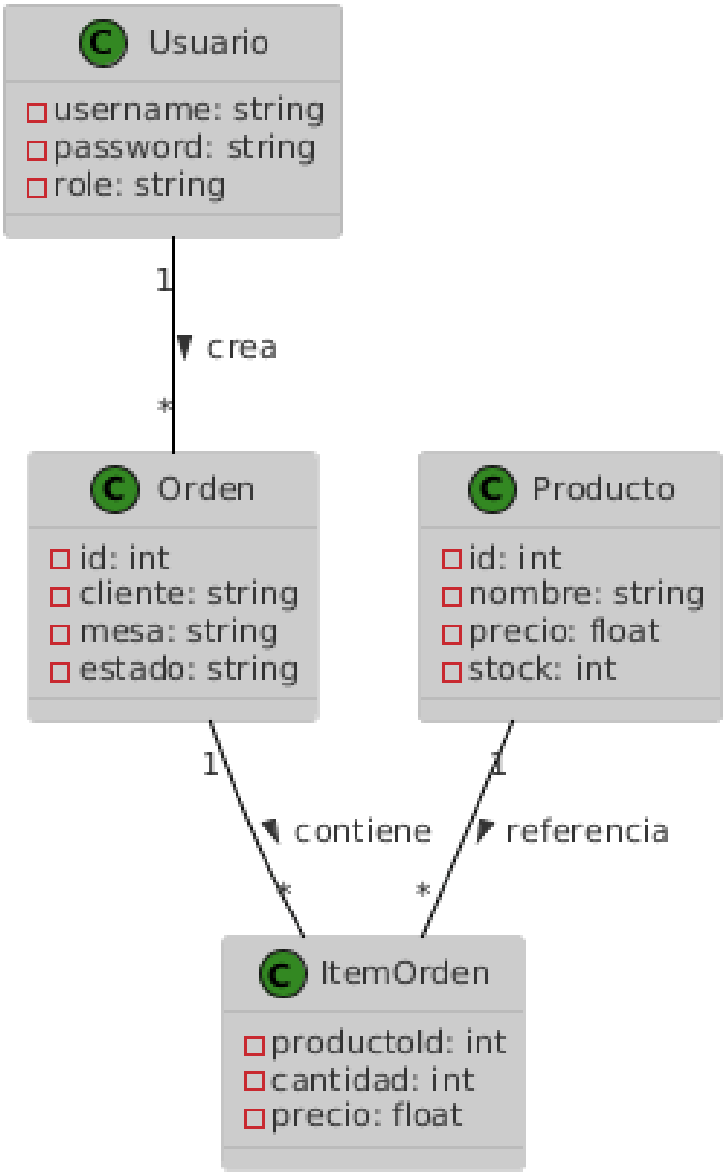
El sistema propuesto permite:

1. Automatizar el flujo de órdenes
2. Reducir errores humanos
3. Mejorar la comunicación entre áreas
4. Controlar inventario en tiempo real
5. Gestionar pagos y estadísticas
6. Escalar a tecnologías más avanzadas

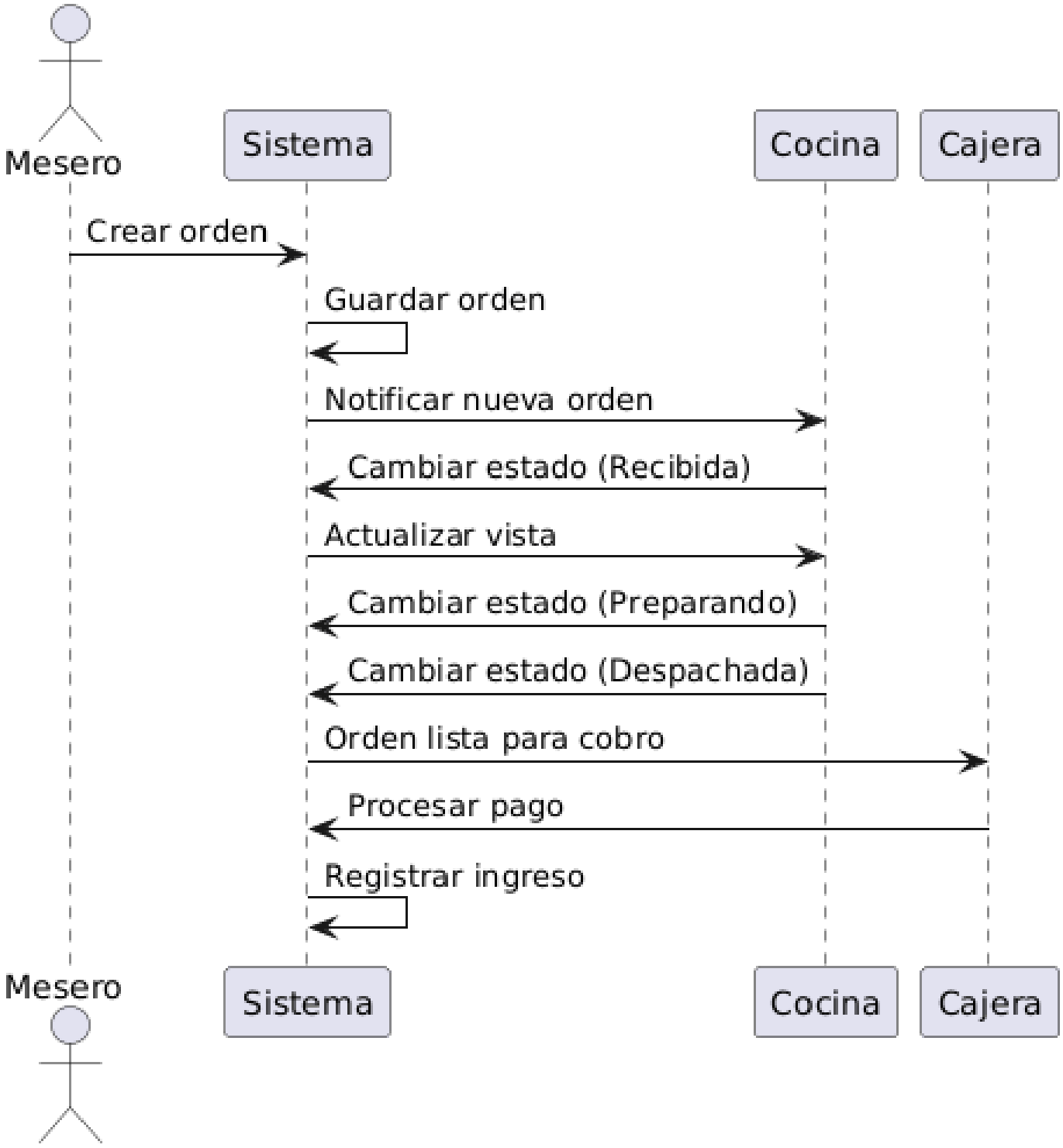
1. DIAGRAMA DE CASOS DE USO



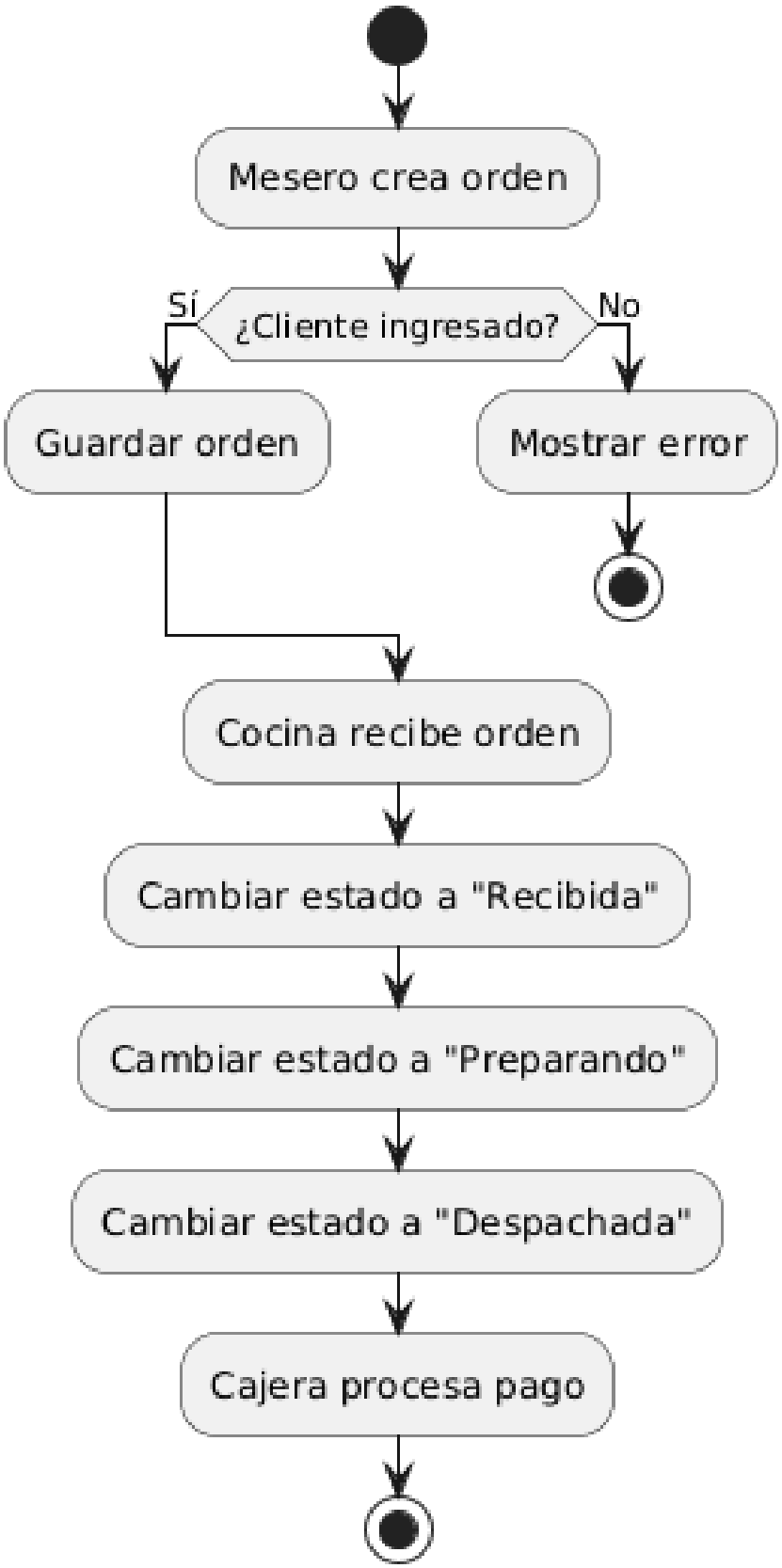
2. DIAGRAMA DE CLASES



3. DIAGRAMA DE SECUENCIA



4. DIAGRAMA DE ACTIVIDAD



5. DIAGRAMA DE ESTADOS

BASE DE DATOS

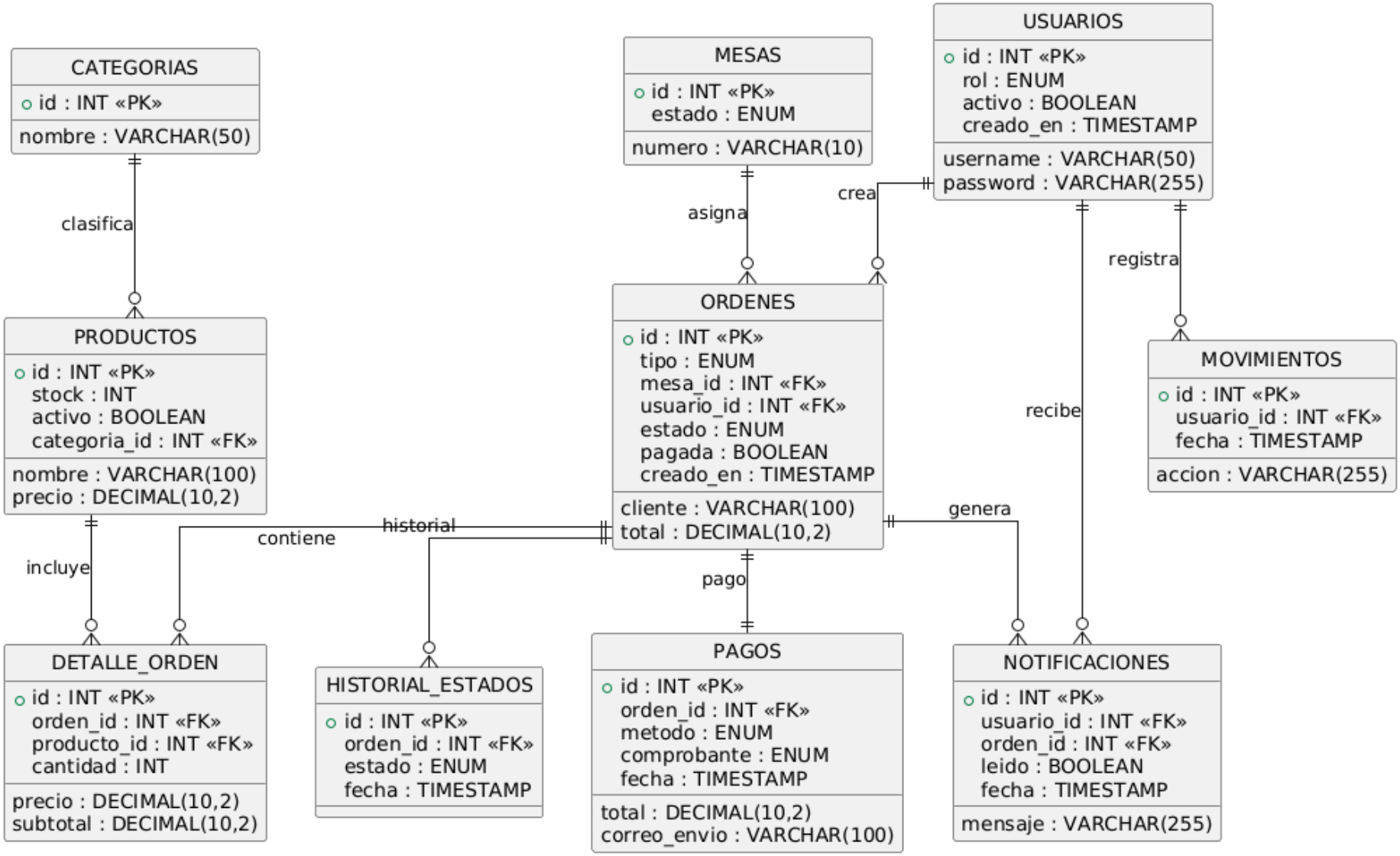
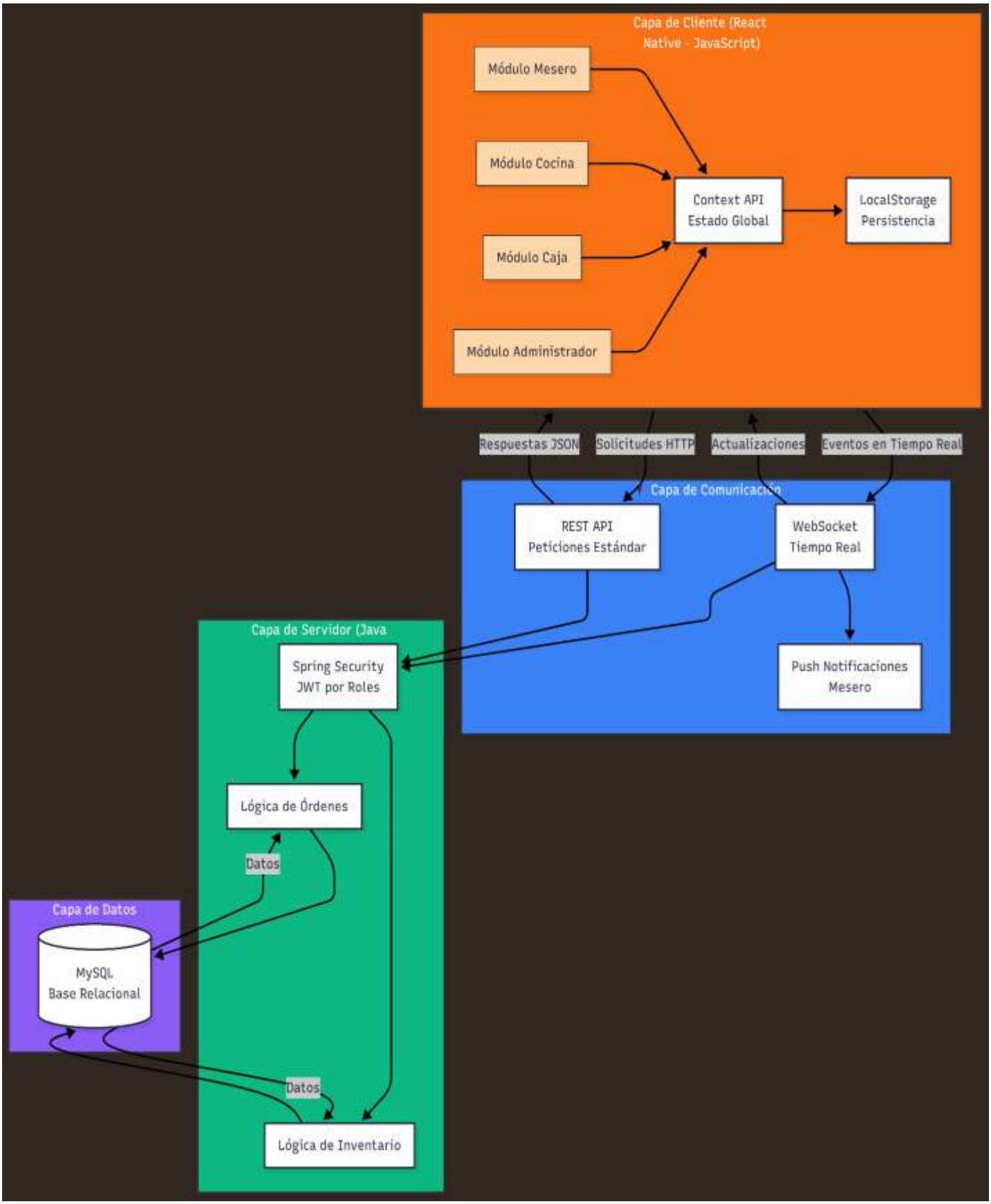


Diagrama gráfico de la arquitectura de software



a. **Descripción de los Componentes de la Arquitectura**

i. Backend y lógica de negocio: la aplicación será desarrollada utilizando el framework **Java Spring Boot**, el cual se encargará de procesar la lógica central del restaurante. Este componente gestionará el ciclo de vida de las órdenes, permitiendo la transición de estados entre Ordenada, Recibida, Preparando y Despachada. Además, controlará las reglas de negocio críticas, como la validación de stock en tiempo real y la restricción de funciones administrativas únicamente para usuarios con dicho rol.

ii. Base de datos y modelo de datos: se implementará **MySQL** como sistema de gestión de bases de datos relacional para garantizar la integridad, consistencia y persistencia definitiva de la información. El modelo de datos se ha diseñado bajo una estructura normalizada que permite una gestión eficiente del flujo de trabajo del restaurante mediante las siguientes entidades principales:

- **Gestión de Accesos:** Tablas de **usuarios** y **roles**, que permiten la autenticación y autorización basada en perfiles (Mesero, Cocina, Caja y Administrador).
- **Catálogo e Inventario:** Tablas de **productos**, **categorias** y **movimientos**, que registran el stock disponible, los precios y las entradas o salidas de insumos controladas por el administrador.
- **Ciclo de Ventas:** La tabla de **mesas** se vincula con la entidad **ordenes** para la asignación de pedidos. El detalle técnico de cada pedido se desglosa en la tabla **detalle_orden**, que conecta los productos seleccionados con su respectiva orden.
- **Trazabilidad y Control:** Se incluye la tabla **historial_estados** para registrar cada cambio en el flujo de la orden (Ordenada, Preparando, etc.) y la tabla **notificaciones** para la comunicación en tiempo real con los meseros.
- **Finalización y Finanzas:** Las tablas de **pagos** y **movimientos** consolidan la información de facturación por mesa, registrando el método de pago (Efectivo, Tarjeta o Transferencia) y el cierre diario de ingresos.

iii. API y protocolos de comunicación La interoperabilidad entre el cliente y el servidor se basará en dos protocolos principales:

- **REST API (HTTP/HTTPS):** Se utilizará para operaciones estándar como el inicio de sesión, la consulta del menú y la creación de nuevos usuarios o productos.
- **WebSockets:** Se implementará para permitir la comunicación bidireccional en tiempo real, facilitando que las órdenes lleguen instantáneamente a la cocina y que los meseros reciban notificaciones push cuando el estado de sus pedidos cambie.

iv. Gestión de sesiones y estado

- **Gestión de Estado del Cliente:** Se utilizará **Context API** en React Native para mantener un estado global que comparta la información del carrito y el usuario activo entre los diferentes módulos.
- **Persistencia:** Se empleará **LocalStorage** para almacenar tokens de acceso y estados temporales, permitiendo que la aplicación se recupere automáticamente ante recargas accidentales o fallos de red sin perder el progreso del usuario.

v. Estrategia de seguridad El sistema implementará una arquitectura de seguridad robusta basada en:

- **Autenticación y Autorización:** Uso de **Spring Security con JSON Web Tokens (JWT)** para validar la identidad de los usuarios y asegurar que cada rol (Mesero, Cocina, Caja, Admin) acceda únicamente a sus funciones correspondientes.
- **Protección de Datos:** Se aplicarán validaciones en los puntos de entrada de la API para prevenir ataques comunes como inyección SQL y asegurar que la comunicación entre dispositivos sea cifrada.

9. Herramientas y tecnologías a utilizar

A. Lenguajes y frameworks.

- Frontend:
    1. React + JavaScript: Desarrollo de interfaces modernas, seguras y mantenibles.
    2. React Router: Navegación entre vistas por rol (mesero, cocina, cajero, administrador).
    3. Tailwind CSS v4: Estilos responsivos y consistentes con la paleta de colores definida.
    4. Lucide React: Iconografía ligera y moderna.

- Backend:
    1. Node.js + Express.js: API REST para gestión de órdenes, usuarios e inventario.
    2. Context API: Manejo de estado global en frontend.
    3. WebSockets (Socket.IO): Comunicación en tiempo real para notificaciones y actualización de estados.

B. Gestor de base de datos.

- MySQL:
    1. Base de datos relacional para la persistencia de usuarios, productos, órdenes y pagos.
    2. Ventajas: Amplia comunidad, soporte institucional, alto rendimiento en consultas transaccionales, integración sencilla con Node.js (mediante librerías como “mysql2” o “sequelize”).
    3. Ideal para manejar relaciones entre tablas (ej. Órdenes ↔ Productos ↔ Usuarios) y garantizar integridad referencial.

- Localstorage (Frontend):
    1. Almacenamiento en cliente para recuperación rápida en caso de fallo o recarga.
    2. Permite mantener el estado de la sesión y restaurar órdenes en curso sin depender del servidor.

- Opcional: Google Sheets API
    1. Para exportación de reportes administrativos en un formato accesible a personal no técnico.
    2. Útil para estadísticas diarias o informes rápidos sin necesidad de acceder directamente a la base de datos

C. Control de versiones Git/Github

Git: Control de versiones distribuido.

GitHub: Repositorio central con ramas organizadas:

- **main:** Versión estable del sistema.
- **develop:** Rama de integración de nuevas funcionalidades.
- **Ramas de desarrollo individuales:** Cada integrante trabaja en una rama con su nombre (ej. edwin, lara, anderson).

Pull Requests y revisiones de código: Aseguran calidad y trazabilidad antes de fusionar cambios en main.

GitHub Issues/Projects: Gestión de tareas, asignación de responsables y seguimiento de avances.

D. Estrategia de CI/CD y herramientas seleccionadas

Integración Continua (CI):

- **GitHub Actions:** Ejecución automática de pruebas unitarias, validación de tipado (JavaScript) y linting en cada push.
- Automatización de builds para asegurar que el sistema se mantenga estable en cada integración.

Despliegue Continuo (CD):

- **Docker:** Uso de contenedores para garantizar portabilidad y consistencia del entorno.
- **PM2:** Gestión de procesos en producción con monitoreo y reinicio automático.

- **Hosting en Railway:** Plataforma utilizada para desplegar el sistema, con soporte para aplicaciones Node.js y bases de datos MySQL, asegurando disponibilidad y escalabilidad.

Monitoreo y logs:

- **Docker logs** y **PM2 monitoreo** para supervisar disponibilidad y rendimiento del sistema.
- Alertas configuradas para detectar caídas o errores en tiempo real.

Estrategia de recuperación:

- Persistencia en **localStorage** para restaurar estado en caso de fallo del cliente.
- Scripts de **backup automático de la base de datos MySQL**, garantizando recuperación de información en caso de incidentes.
- Procedimientos de restauración documentados para minimizar tiempos de inactividad.

Licencias Creative Commons (Descripción para Proyecto Académico)

Las **Licencias Creative Commons (CC)** son un conjunto de permisos legales que permiten a los autores decidir cómo se puede utilizar, compartir o modificar su obra. Estas licencias facilitan el acceso al contenido, fomentan la colaboración y garantizan reconocimiento al creador original.

Creative Commons no reemplaza los derechos de autor, sino que los **complementa**, permitiendo al creador definir qué usos están permitidos sin necesidad de autorizar cada caso individualmente.

Licencia seleccionada: CC BY-NC-ND

(**Atribución – No Comercial – Sin Derivadas**)

1. PROHÍBE el uso comercial sin permiso

Como tu app **simula que será vendida**, esta licencia evita que terceras personas:

- Usen tu software en negocios reales.
- Lo monetizen.
- Lo incorporen en productos propios con fines de lucro.

Solo tú, como autor, mantienes el derecho a comercializarlo.

2. PROHÍBE que modifiquen tu proyecto

No permite que otros:

- Cambien el código
- Modifiquen el diseño
- Hagan versiones alternativas
- Reutilicen partes de tu trabajo como una base para otro software

Esto es clave si planeas **venderlo, licenciarlo o proteger tu idea**.

3. PROHÍBE que lo reproduzcan o redistribuyan libremente

Nadie puede publicarlo, compartirlo o difundirlo sin tu autorización expresa.

La única acción permitida es **verlo o analizarlo con fines educativos**, siempre con atribución.

4. Asegura que te den crédito como autor

Cualquier persona que hable del proyecto debe mencionarte como creador, pero no tiene derecho a usarlo.

Justificación:

La elección de la licencia **Creative Commons BY-NC-ND** se realiza con el objetivo de proteger la integridad y la futura explotación comercial del proyecto. Dado que este sistema multiplataforma simula una aplicación destinada a ser vendida, es necesario restringir completamente su uso, reproducción, modificación y distribución sin autorización.

Esta licencia garantiza que:

- **Ningún tercero pueda utilizar el software con fines comerciales**, preservando así el valor económico del proyecto.
- **No se generen versiones derivadas**, evitando alteraciones no autorizadas o apropiación de elementos funcionales y visuales.
- **No pueda ser redistribuido**, manteniendo el control total sobre su difusión.

Al mismo tiempo, permite mostrar el proyecto de manera académica siempre que se reconozca la autoría original. Con ello, se asegura la protección intelectual mientras se cumple con los requerimientos de la materia y la exposición del proyecto.

Presupuesto

Recursos Humanos

Rol	Actividad	Horas estimadas	Costo por hora	Subtotal
Desarrollador Principal	Implementación general, integración, pruebas	40 h	$10/h	**$400**
Diseñador UI/UX	Prototipado en Figma, diseño de pantallas	10 h	$8/h	**$80**
Desarrollador Frontend	Componentes, rutas, UI React	20 h	$9/h	**$180**
Desarrollador Backend	API, base de datos, lógica	20 h	$10/h	**$200**
Tester / QA	Pruebas funcionales y reporte de errores	10 h	$7/h	**$70**

✓ **Subtotal Recursos Humanos: $930**

Infraestructura Tecnológica

Recurso	Detalle	Costo mensual	Meses	Subtotal
Hosting (VPS o Shared)	Para backend y base de datos	$10	1	**$10**
Base de datos (si se usa servicio externo tipo Railway/Supabase)	Plan básico	$5	1	**$5**
Dominio (opcional)	.com o .sv	$12 anual	Proporción 1 mes ($1)	**$1**

✓ **Subtotal Infraestructura: $16**

Software y Herramientas

Herramienta	Función	Costo
Trello	Gestión Kanban del proyecto	**$0 (plan gratuito)**
GitHub	Control de versiones y repositorio	**$0**
Figma	Diseño UI/UX	**$0**
Visual Studio Code	Entorno de desarrollo	**$0**

✓ **Subtotal Software: $0**

Equipos y Materiales

Material o equipo	Uso	Costo aproximado
Energía eléctrica y datos (proporcional)	Desarrollo, pruebas	**$10**
Material de apoyo / impresiones	Documentos del proyecto	**$5**

✓ **Subtotal Equipos y Materiales: $15**

Contingencias (10%)

Para cubrir imprevistos técnicos o extensiones de módulos:

- **10% del total parcial ($930 + $16 + $15) = $96**

✓ **Subtotal Contingencias: $96**

Resumen General del Presupuesto

Categoría	Subtotal
Recursos Humanos	$930
Infraestructura	$16
Software	$0
Equipos y Materiales	$15
Contingencias	$96

TOTAL ESTIMADO DEL PROYECTO: $1,057 USD

Cronograma de trabajo

Fase 1 – Análisis y Planificación (Semana 1)

Actividades

- Revisión del problema y requerimientos funcionales
- Definición del alcance del sistema
- Análisis del prototipo de Figma
- Configuración del tablero Kanban en Trello
- Creación del backlog inicial de tareas

Duración: 1 semana

Fase 2 – Diseño del Sistema (Semana 2)

Actividades

- Ajustes del prototipo de Figma según requerimientos
- Arquitectura del sistema (componentes React, Context API, rutas)
- Diseño del flujo de navegación (router)
- Definición del modelo de datos y endpoints de API

Duración: 1 semana

Fase 3 – Desarrollo del Módulo de Autenticación (Semana 3)

Actividades

- Implementación del *AuthProvider*
- Pantallas: login, registro, recuperación
- Validaciones y control de acceso
- Pruebas iniciales del flujo de autenticación

Duración: 1 semana

Fase 4 – Desarrollo del Módulo de Menú / Platillos (Semana 4 y 5)

Actividades

- CRUD de platillos (crear, listar, editar, eliminar)
- Administración de categorías
- Integración visual del diseño de Figma
- Pruebas funcionales del módulo

Duración: 2 semanas

Fase 5 – Módulo de Pedidos y Gestión de Mesas (Semana 6 y 7)

Actividades

- Registro de pedidos
- Envío de pedidos a cocina
- Estado del pedido (pendiente, en preparación, entregado)
- Control de mesas ocupadas / disponibles
- Integración con notificaciones (*Toaster*)

⏱ **Duración: 2 semanas**

Fase 6 – Módulo Administrativo e Inventario (Semana 8)

Actividades

- Panel de administración
- Gestión de usuarios y roles
- Control de inventario básico
- Generación de reportes simples

Duración: 1 semana

Fase 7 – Pruebas, Correcciones y Optimización (Semana 9)

Actividades

- Pruebas completas del sistema
- Corrección de errores
- Optimización de componentes
- Revisión del manejo de errores con *ErrorBoundary*
- Revisión de flujos en Trello y documentación de tareas

Duración: 1 semana

Fase 8 – Documentación y Entrega Final (Semana 10)

Actividades

- Preparación del documento final del proyecto
- Capturas del sistema y evidencia en Trello
- Elaboración de la presentación final
- Ensayo de defensa del proyecto

Duración: 1 semana

Resumen del Cronograma

Fase	Actividades principales	Duración
1	Análisis y Planificación	1 semana
2	Diseño del Sistema	1 semana
3	Módulo de Autenticación	1 semana
4	Módulo de Menú	2 semanas
5	Pedidos y Mesas	2 semanas
6	Panel Admin e Inventario	1 semana
7	Pruebas y Optimización	1 semana
8	Documentación	1 semana

Duración Total: 10 Semanas

Fuentes de consulta

- ✓ Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
- ✓ Figma. (2026). *Figma: The collaborative interface design tool* (Versión 124.3.2). https://www.figma.com
- ✓ Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- ✓ Google. (2026). *Material Design 3: Guide for Android and Web development*. https://m3.material.io
- ✓ Oracle Corporation. (2023). *MySQL documentation*. https://dev.mysql.com/doc/
- ✓ Microsoft. (2023). *Database design documentation*. https://learn.microsoft.com/
- ✓ Meta Platforms, Inc. (2023). *React documentation*. https://react.dev
- ✓ PlantUML. (2023). *PlantUML language reference guide*. https://plantuml.com/
- ✓ Tailwind Labs. (2023). *Tailwind CSS documentation*. https://tailwindcss.com
- ✓ Anderson, D. J. (2010). *Kanban: Successful evolutionary change for your technology business*. Blue Hole Press.
- ✓ Atlassian. (2024). *Trello guide*. https://trello.com/guide
- ✓ Atlassian. (2024). *Trello user guide*. https://support.atlassian.com/trello/
- ✓ Atlassian. (2024). *What is Kanban?* https://www.atlassian.com/agile/kanban

- ✓ Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice*. Addison-Wesley.

- ✓ Benson, J., & Barry, T. D. (2011). *Personal Kanban: Mapping work, navigating life*. Modus Cooperandi Press.

- ✓ Creative Commons. (2024). *About the licenses*. https://creativecommons.org/licenses/

- ✓ Creative Commons. (2024). *Choosing a license*. https://creativecommons.org/choose/

- ✓ Creative Commons. (2024). *FAQ*. https://creativecommons.org/faq

- ✓ Figma Inc. (2024). *Figma design documentation*. https://help.figma.com
- ✓ Garrett, J. J. (2011). *The elements of user experience*. New Riders.

- ✓ Interaction Design Foundation. (2024). *User interface (UI) design basics*. https://www.interaction-design.org/

- ✓ Kowal, E. (2024). *Sonner: Toast notifications for React*. https://sonner.emilkowal.ski/

- ✓ Kniberg, H., & Skarin, M. (2010). *Kanban and Scrum: Making the most of both*. C4Media.

- ✓ Larman, C. (2004). *Applying UML and patterns*. Prentice Hall.

- ✓ Lessig, L. (2004). *Free culture: How big media uses technology and the law to lock down culture*. Penguin Press.

- ✓ Meta Open Source. (2023). *React Context API overview*. https://reactjs.org/docs/context.html (reactjs.org in Bing)

- ✓ Mozilla Developer Network. (2024). *JavaScript documentation*. https://developer.mozilla.org/

- ✓ Norman, D. (2013). *The design of everyday things*. MIT Press.

- ✓ Pressman, R. S. (2014). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill.

- ✓ Project Management Institute. (2017). *Agile practice guide*. PMI.

- ✓ React Router Team. (2024). *React Router documentation*. https://reactrouter.com

- ✓ React Team. (2024). *React documentation*. https://react.dev

- ✓ Trello. (2024). *How to use Kanban with Trello*. https://trello.com

- ✓ Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.